Technical Specifications
For PukParking

Arno Eybers - 50221493
Carlo Botha - 50824309
Celumusa Mkhatshwa - 51800896
Marco Goncalves - 45614237
Delphy Stone - 50509772
Heinrich de Laan - 35159073
Rieghardt Wehmeyer - 42262186
Phetogo Monaisa - 29909767

Table of Contents:

1. Introduction **3**
- 1.1 Purpose of the system 3
- 1.2 Abbreviations and acronyms used 3

2. Overall Description **4**
- 2.1 Product Perspective 4
- 2.2 User Classes and Characteristics 4
- 2.3 Basic Technical Features 4
- 2.4 Operating environment 5
- 2.5 Design Constraints 5
- 2.6 Assumptions and Dependencies 5

3. System Architecture **6**
- 3.1 High-Level Architecture 6
- 3.2 High-Level Architecture 7
- 3.3 Data Flow Diagram 8

4. Technology Stack and Tools **10**
- 4.1 Frontend 10
- 4.2 Backend 11
- 4.3 Map and Geolocation Technology 13

5. API Information **14**
- 5.1 Purpose of APIs 14
- 5.2 Why Mapbox API Was Selected 14
- 5.3 Benefits of Mapbox for PukParking 15

6. Pricing and Software procurement **16**
- 6.1 Overview 16
- 6.2 Mapbox Costs 16
- 6.3 Database Costs 17
- 6.4 Backend Hosting 18
- 6.5 Frontend Hosting 18
- 6.6 Source Code Management 19
- 6.7 Software Procurement Summary 19
- 6.8 Hardware Requirement 20

7. External Interface Requirements **21**
- 7.1 User Interfaces 21
- 7.2 Software Interfaces 27
- 7.3 Communication Interfaces 29

8. Wireframes **30**

9. References **31**

1. Introduction

1.1 Purpose of the system

PukParking is a progressive web application (PWA) designed to provide real-time, destination-based parking recommendations and walking navigation for North-West University (Potchefstroom Campus) students, staff, and visitors without requiring user authentication. For more detailed information, please refer to the PukParking Functional Requirements Specification (FRS).

1.2 Abbreviations and acronyms used

Abbreviation	**Definition / Expansion**
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DB	Database
DBMS	Database Management System
ES6+	ECMAScript 6+ (Modern JavaScript Standard)
FRS	Functional Requirements Specification
GIS	Geographic Information System
GL JS	Graphics Library JavaScript
GPS	Global Positioning System
HTML5	HyperText Markup Language 5
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JS	JavaScript
JWT	JSON Web Token
LTS	Long-Term Support
MIME	Multipurpose Internet Mail Extensions
NWU	North-West University
ORM	Object-Relational Mapping (or Object-Document Mapping via Mongoose)
POPIA	Protection of Personal Information Act
PWA	Progressive Web Application
REST	Representational State Transfer
UI	User Interface
URL / URI	Uniform Resource Identifier / Locator (Implied via REST API routes)
WiFi	Wireless Fidelity
PII	Personally Identifiable Information

2. Overall Description

2.1 Product Perspective

PukParking is a standalone, mobile-first PWA that intends to eliminate time wasted while searching for a parking spot by guiding users directly to a valid, available parking area relatively close to their target destination on campus.

As established in the Feasibility Study, a custom-built PWA (scoring 8.85/10) was selected over the other two discussed options. PukParking will be using Sensor-less crowdsourced data combined with spatial pathfinding algorithms to provide a low-cost, scalable platform tailored specifically to NWU disc/zone rules without expensive hardware overhead.

2.2 User Classes and Characteristics

As stated in the Functional Specifications document, PukParking has only two access levels:

1. **Public Users**: The public interface does not require an account and is thus shared by all types of people on campus. Different people have different parking permissions, so the system must allow the applicable parking category to be selected or applied when filtering recommendations.
2. **Authenticated Administrators:** Administrators use a protected management interface to maintain the campus data used by the public application.

2.3 Basic Technical Features

2.3.1 Administrator Authentication Subsystem

Secure management portal utilizing JWT bearer tokens and bcrypt password hashing to authenticate administrative staff and safeguard REST API endpoints for managing building and parking data.

2.3.2 Data Storage Subsystem

MongoDB documents database storing campus building details, parking area attributes (such as coordinates, capacities, occupied space estimates, operating hours, and zone restrictions), administrator credentials, and temporary parking full reports.

2.3.3 Mapping & Navigation Engine

Mapbox GL JS client-side rendering paired with Mapbox Directions and Geocoding APIs for location searching and walking route generation, using the Haversine formula for distance calculations and the native browser Geolocation API to detect current positioning.

2.3.4 Crowdsourced Status & Reporting

Login-free, anonymous reporting mechanism that lets public users submit "Parking Full" signals, temporarily increasing occupancy confidence scores and re-prioritizing recommendations without collecting personal data or tracking user identities.

2.4 Operating environment

- Client Side: Any standard, modern mobile or desktop web browser (Chrome, Safari, Firefox, Edge) supporting PWA Service Workers, JavaScript (ES6+), and HTML5 Geolocation.
- Server Runtime Environment: Node.js (v18+ LTS) environment executing an Express.js REST API framework.
- Database Environment: MongoDB Instance (v6.0+) managed locally or through MongoDB Atlas.

2.5 Design Constraints

POPIA Compliance (Data Privacy): The application must operate without requiring end-user registration or tracking individual movement history to avoid storing personal identifiable information (PII).

Technology Stack Constraints: Built exclusively using open-source libraries or standard platform offerings (React, TypeScript, Express, Node.js, Tailwind CSS) without reliance on proprietary paid frameworks outside standard cloud hosting.

Version Control: Source code must be version-controlled in the official project GitHub repository, enforcing modular feature-branch workflows.

2.6 Assumptions and Dependencies

2.6.1 Assumptions

Users have compatible devices with internet access and GPS capabilities enabled to display location-aware routing.

Users are familiar with English to an acceptable degree.

2.6.2 Dependencies

System uptime and route rendering depend on third-party service availability, specifically Mapbox APIs (Mapbox GL JS, Directions API). Server infrastructure depends on cloud-hosted Node.js runtime availability and MongoDB database uptime.

Reliable Wi-Fi or cellular connectivity for public users to access the web application, load maps, submit live reports, and refresh routes.

Reliance on public users to submit "Parking Full" reports to maintain temporary live availability metrics, alongside manual administrative updates for campus records.

3. System Architecture

3.1 High-Level Architecture

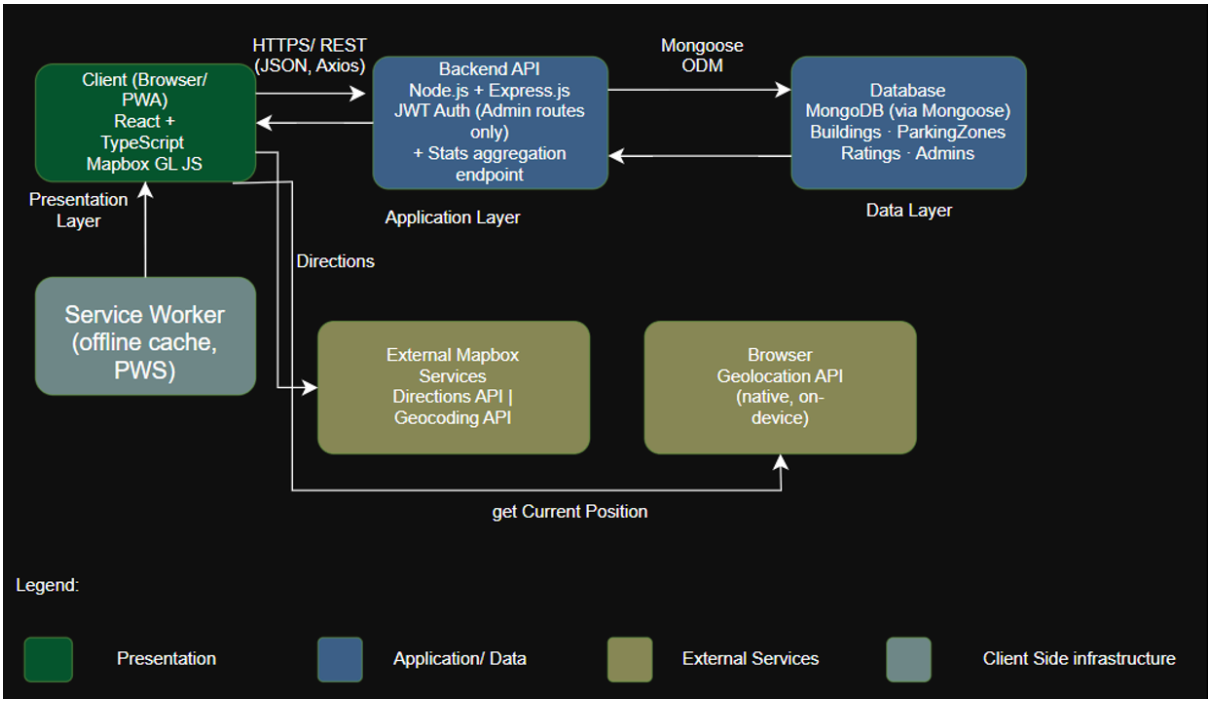
Pukparking follows a three-tier client-server architecture, dividing the system into a presentation layer, an API layer and a data layer. This division allows for the independent development, testing and deployment of the frontend and the backend. This is suited for how work is divided within the team.

The presentation layer is a React single page application, built as a Progressive Web App so it can be installed to a device home screen and continues to function partially via a service worker. This is for the case where users navigate on campus where there is inconsistent WIFI or mobile data coverage.

The application layer is a Node.js/ Express REST API. This is the only component permitted to directly communicate directly with the database and it enforces authentication on all administrator-only endpoints. All communication between the client and the API occurs over HTTPS, with data exchanged as JSON.

The data layer is a MongoDB database, accessed through Mongoose, which stores parking zone and spot records, and administrator accounts.

Two categories of external service sit outside PukParking's own three tires: the Mapbox platform (map rendering, walking directions and location search) and the browser's native Geolocation API (used to determine the user's current position on-device, without any data leaving the browser)



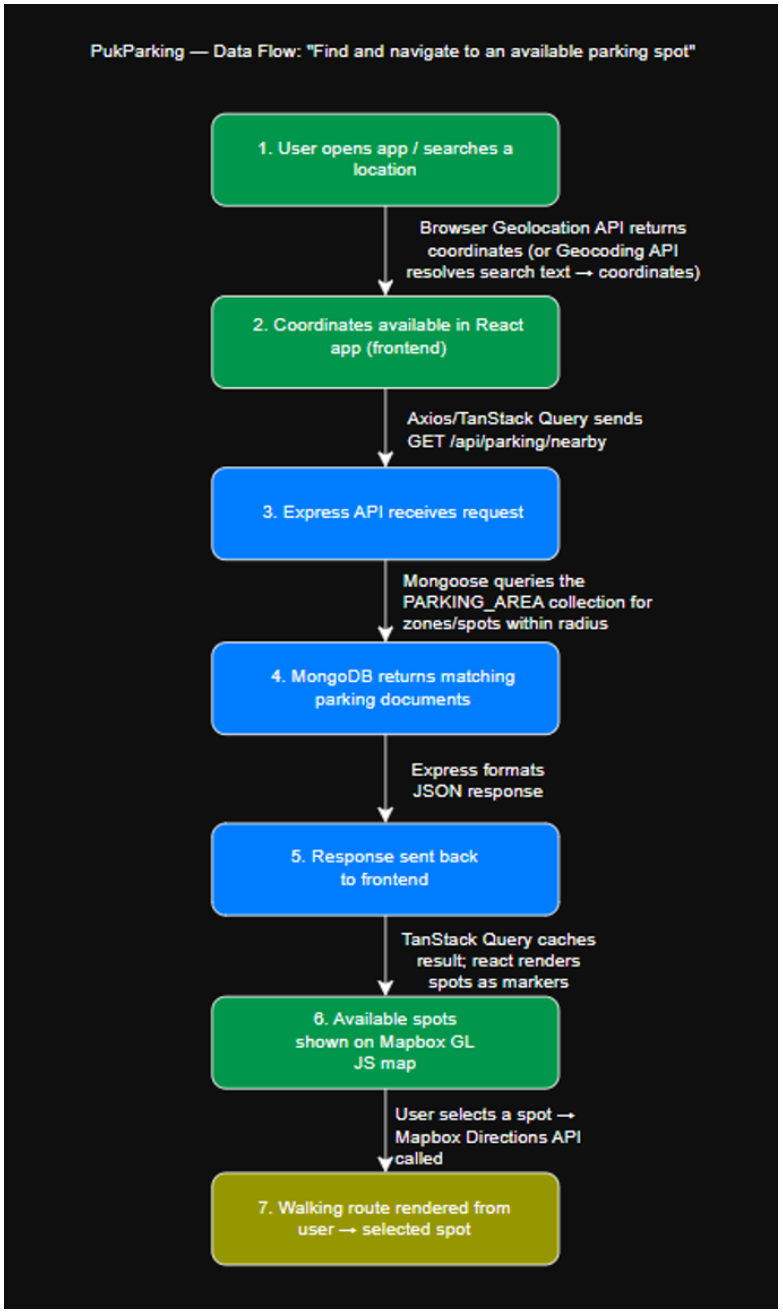
3.2 High-Level Architecture

This section covers the reasoning behind key architectural decisions made for PukParking, rather than simply listing what was chosen.

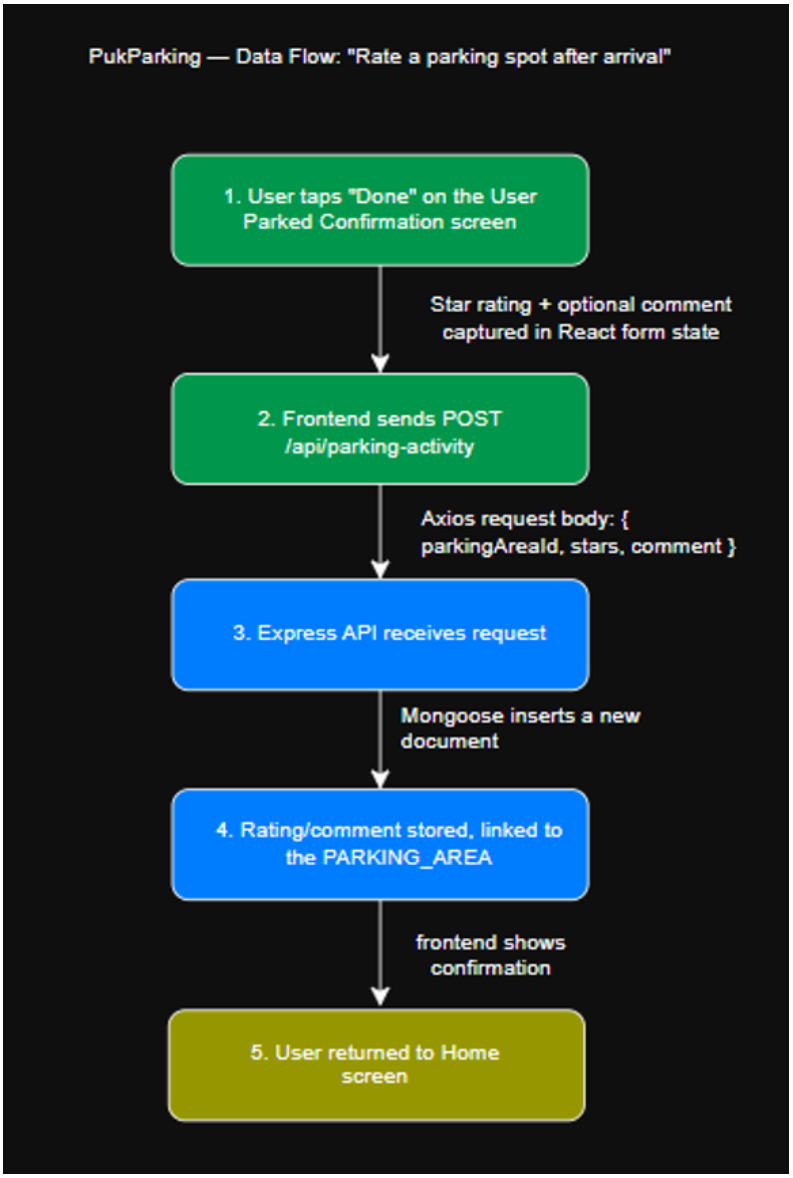
- **Client-server REST architecture over a monolith:** A REST API allows for clean distinction between front end and backend concerns, lets each tier scale or redeploy independently, and mathes how the team divided the work (frontend/ backend sub-teams). Both tiers are written in JavaScript/TypeScript, so the language is consistent between the two tiers and contributors don't have to use a second language.
- **MongoDB as the database:** Parking zone and spot data varies across campus locations (different attributes per zone, evolving fields during development), and MongoDB's document model accommodates without the requirement of a rigid schema to be established upfront. Mongoose is layered on top to specifically reintroduce schema validation where needed, balancing flexibility with data integrity.
- **JWT for authentication:** Only administrators need authentication (regular users can use without an account), so a stateless token based approach avoids the overhead of server-side session storage and keeps the API layer simpler to scale. The token is issued on login and validated by Express middleware on every protected route.
- **Mapbox over alternatives:** Mapbox GL JS renders vector tiles, giving a smoother pan/zoom and combines Directions and Geocoding APIs into one platform/API key rather than separate integrations.
- **Progressive Web App over a native app:** A PWA can be installed directly from the browser with no app store submission or review process, which makes it fit for this project.
- **Browser Geolocation API over a third-party location service:** It is native to the browser, free, and requires no additional dependency. This is sufficient for PukParking's accuracy needs.

3.3 Data Flow Diagram

Find and navigate to an available parking spot



Rate a parking spot after arrival



4. Technology Stack and Tools

4.1 Frontend

Technology	Purpose	Rationale
React	Build the user interface	Because the UI uses reusable components and repeating UI patterns, React assists in making the developer experience more pleasant.
TypeScript	Strong typing and improved maintainability	Unlike javascript, typescript adds type related errors due to it's strict type checking, this can minimize bugs
Vite	Development server and production build tool	Because React code isn't natively understood by the browser and has to be compiled and bundled into plain JS, Vite allows you to do this fast.
Tailwind CSS	Responsive styling and UI design	Tailwind CSS allows for the use of utility classes instead of writing custom CSS files. It speeds up building.
React Router	Navigation between application pages	Handles moving between pages without full browser reload.
Axios	Communication with backend APIs	Makes HTTP requests to the Express API. For communication between the frontend and the backend.
Vite PWA Plugin	Progressive Web App support	Enables PukParking to be installed to a home screen and used offline or on poor connectivity, relevant for students checking availability on campus Wi-Fi or mobile data.

Service Workers	Offline caching and faster loading	Underpins the PWA functionality above; caches static assets and recent parking data so the app remains partially usable without a live connection.

4.2 Backend

Technology	Purpose	Rationale
Node.js	JavaScript runtime	Consistent with the typescript used in the frontend. Node.js is the javascript runtime the whole backend runs on. It will allow the backend logic to be written in the same language as the frontend.
Express.js	REST API	Because raw Node.js has no built-in routing system, Express turns it into a REST API(routes, middleware, request/ response handling).
MongoDB	Store application data	Document-based schema fits semi-structured data such as parking zone metadata and varying attributes across campus locations, without requiring a rigid predefined schema during early iterations.
Mongoose	MongoDB object modelling	Adds schema validation and structure on top of MongoDB's flexibility, reducing the risk of malformed documents while retaining schema flexibility.
JWT	Administrator authentication	Stateless authentication with no server-side session storage required, simplifying scaling and deployment for a small-team project.

bcrypt	Password hashing	Industry-standard, salted password hashing. Protects admin credentials even if the database is compromised.
dotenv	Environment variable management	Keeps secrets (API keys, DB connection strings) out of source control, satisfying basic security hygiene for a GitHub-hosted repository.
Multer	Upload parking images	Handles multipart/form-data uploads needed for admin-submitted images of parking zones and signage.
CORS	Enable frontend/backend communication	Required because frontend and backend are hosted separately; explicitly configured to permit only the deployed frontend origin, rather than left open.
Helmet	HTTP security headers	Sets secure HTTP headers (e.g. against clickjacking, MIME-sniffing) with minimal configuration effort, a lightweight security win.

4.3 Map and Geolocation Technology

Technology	Purpose	Rationale
Mapbox GL JS	Interactive campus map	Javascript library that actually draws and renders interactive map in the browser. This is consistent with the javascript used in the rest of the backend and the typescript used in the frontend.
Mapbox Directions API	Walking route generation	Provides turn by turn directions between two points
Mapbox Geocoding API	Location searching	Converts a text search into coordinates and can also do the reverse.
Browser Geolocation API	Detect user's current location	Native browser API with no third-party dependency, used to centre the map on the user and calculate proximity to available spots.

5. API Information

5.1 Purpose of APIs

The API is essential for enabling various functions to interact and exchange data. As for the mapping and navigation functions of PukParking, it makes use of the mapping information, geographic information system (GIS) and routing information provided by APIs instead of developing and managing these resources by itself.

We will use the Mapbox platform for the mapping and geocoding-related API calls. For the campus map, we will be using Mapbox GL JS to draw the map and add interactive features to it within the frontend. For the walking routes, we will be using the Mapbox Directions API and for the location searching, we will be using the Mapbox Geocoding API. The browser's native Geolocation API will also be used to retrieve the current location of a user.

5.2 Why Mapbox API Was Selected

Mapbox was chosen because PukParking is largely a location-based system. Users will view maps of campus with parking locations marked. They will search for locations by name and find shortest paths as a walking route to a chosen parking area. The campus map can be viewed in an interactive Mapbox GL JS map within the browser. Mapbox describes GL JS as a "JavaScript library for interactive web maps", which is ideal for PukParking's frontend built with React in combination with TypeScript (Mapbox, 2026a). (Mapbox)

For the map displayed on the browser's screen, PukParking uses Mapbox GL JS (Mapbox, 2026a). (Mapbox) GL JS is a JavaScript library for interactive web maps (Mapbox, 2026a). (Mapbox) It can render maps on which users can zoom and pan. On the PukParking campus map this enables users of the system to interact with the map that they view.

The **Mapbox Directions API** is also used in the application to generate walking routes between locations. While identifying a suitable parking space is the main objective of PukParking, helping the user to get to the chosen parking location is equally important. As mentioned before, the Mapbox Directions API supports not only walking but also cycling and driving routes. Since Mapbox states that "routing lets your app tell users where to go," the API is perfect for PukParking's objective (Mapbox, 2026b). (Mapbox)

Mapbox Geocoding API – is used to find locations that match user input in search field. Mapbox Geocoding API provides forward and reverse geocoding features. PukParking could use these features to associate human readable campus location names (e.g." Student Union Building ") with their corresponding geographic coordinates (e.g. lat, lon).

The **Browser's native Geolocation API** is used in conjunction with the functionality offered by the Mapbox services. Whilst the API offered by Mapbox could be used in its entirety to gather the location of the user, this is not necessary for the PukParking system, as the browser itself is able to offer this functionality already.

5.3 Benefits of Mapbox for PukParking

The use of Mapbox provides several benefits:

1. **Reduced development complexity.** By using a mapping API, the development team does not have to set up and maintain a complete mapping and routing solution in order to provide an interactive campus map.
2. **Interactive maps** – Mapbox GL JS offers an interactive web mapping feature to the PukParking's React frontend.
3. **Walking navigation** – The Directions API allows the application to provide routes from a user's location to a selected parking area.
4. **Location searching** – Geocoding allows textual locations to be converted into geographic coordinates.
5. **Scalability** – Mapbox is designed to be used on a huge scale and the service is charged for on a usage basis so a small application such as PukParking will initially operate within the free usage limits and only incur costs as usage increases.
6. **Integration with existing technology stack** – Mapbox GL JS is a JavaScript library written in the programming language, therefore a natural integration with frontend code written in React and backend code written in TypeScript.

The current free limits for Mapbox are 50,000 Map Loads for Web per month, 100,000 Requests for the Directions API per month and 100,000 Requests for the Temporary Geocoding API per month. The cost after this for the various APIs is US$5 per 1,000 Map Loads for Web (Mapbox, 2026d), US$2 per 1,000 Requests for the Directions API (Mapbox, 2026d) and US$0.75 per 1,000 Requests for the Temporary Geocoding API (Mapbox, 2026d).

Mapbox's free tiers will likely be enough for an initial campus deployment and allow for small numbers of students to use the system without Mapbox incurring any charges whilst in development.

6. Pricing and Software procurement

6.1 Overview

The PukParking system is built from open-source software and cloud services, so initially there is no software cost as all software is released under Open-Source licenses. We will, however, incur costs for services like API calls, hosting and domain registration as well as a database in the cloud.

The primary cost for PukParking however will be for the external services that are used, the hosting for the system, the databases and the domain name, as well as API usage.

Note: For comparison purposes we have used an approximate exchange rate of R16.25/US$1. We will update the rates when it is time to procure the product.

6.2 Mapbox Costs

There is no change in Mapbox's current pricing model – it is still a usage-based model. The relevant allowances for the services that PukParking will use are shown below.

Service	**Current free allowance**	**Indicative price after free tier**
Mapbox GL JS – Web Map Loads	50,000 loads/month	US$5 / 1,000 loads
Directions API	100,000 requests/month	US$2 / 1,000 requests
Temporary Geocoding API	100,000 requests/month	US$0.75 / 1,000 requests

Note that these allowances are updated annually. For current pricing information see Mapbox (2026d). (Mapbox, 2026d).

Using the rate above for calculation purposes: R16.25/US$, this translates into R81.25 per 1,000 additional map loads for the service (or R20.31 per additional map load).

- Map Loads: R81.25 per 1,000 additional loads
- Directions: R32.50 per 1,000 additional requests
- Geocoding: R12.19 per 1,000 additional requests

Provided the PukParking system does not exceed the respective free allowances offered by Mapbox for the various services, there will be no cost involved for these services to operate on behalf of the PukParking system, i.e. that there will be no cost

involved for these services to operate on behalf of the PukParking system. If usage does exceed the free thresholds, cost will kick in only as and when billable requests or map loads occur. This cost is expected to increase with each additional request or map load made.

Mapbox GL JS loads a Mapbox map when the Mapbox GL JS HTML map code is added to a webpage. Subsequent interactions with the map after it has loaded do not count as additional map loads for the session (Mapbox, 2026a).

6.3 Database Costs

PukParking is a web application built to manage various parking spaces at different university campuses. Since information of various parking spaces is semi-structured, the application uses MongoDB DBMS (database management system) for its backend data storage. In addition to this, the application also uses Mongoose to add more schema validation to the database objects (document). For the development of the PukParking application, the developer would use MongoDB Atlas Free.

For development purposes such as the prototype proposed in this specification, MongoDB Atlas Free can be used for the database as it offers a free Atlas cluster that costs nothing per hour and has 512 MB of storage. Such a DBMS is intended for, amongst others, learning, development and smaller-scale applications (MongoDB, 2026a). (MongoDB)

For production deployment the database may need to move to a paid option. Currently MongoDB are listing their Flex option at around $0.011 per hour so that would work out at around R0.1885 per hour and so up to around R488 per month. The dedicated option starts at $56.94 per month for a variety of configurations (MongoDB, 2026a). (MongoDB)

Using R16.25/US$1 for estimation:

Database option	***Approximate monthly cost***
MongoDB Atlas Free	*R0*
MongoDB Atlas Flex	*Up to approximately R488/month*
Dedicated Atlas	*From approximately R925/month*

This means for a small proof of concept or development the free option is sufficient. As a result, for PukParking to be university-wide, use the option of MongoDB Atlas Flex or even a fully dedicated MongoDB Atlas host will have to be considered.

6.4 Backend Hosting

In the technology specification above we described how we used Express.js as the REST API layer. That means for backend hosting we are looking for a Node.js hosting platform. There are many hosting platforms that support Node.js and Express.js for web services. Render is one such platform which also supports deployment of apps from Git repositories. Note that Render also supports free web-service instances which we can use for testing and development.

Render is the preferred option to host the PukParking backend written in Node.js and Express web services. Render supports Node.js as a JavaScript runtime environment and Express web services to create the REST API layer in PukParking. To deploy the PukParking backend written in Node.js and Express web services, rendering can be done from a Git repository (Render, 2026a). The PukParking backend can be deployed using free Rendering instances while testing and developing the PukParking backend. However, the free instances are not for production and should not be used for production backends (Render, 2026a).

To host a production solution for the back end of PukParking, Render's paid web-service instances will start at approximately R114.00/month (Render, 2026b).

Compared to the cost of buying and maintaining a server or cluster of servers, cloud hosting provides cost effective hosting for the initial implementation of PukParking.

6.5 Frontend Hosting

The frontend – compiled into a static website by the build tools – can then be deployed to services such as Render or Cloudflare Pages. This type of service is free for static websites hosted by the author at Render and are supported by the Software Procurement component.

Cloudflare Pages will host the static compiled frontend for PukParking. It's free hosting for static assets and there's no charge for requests to static assets. We can deploy the application using Git and it supports custom domain names too (Cloudflare, 2026a). (Cloudflare Docs)

However, the frontend hosting for static assets can be hosted for R0 for the initial phase, given it is a static website and can be uploaded to hosting platforms such as Cloudflare Pages.

6.6 Source Code Management

GitHub can be used for source-code management, collaboration and version control. The GitHub Free plan is currently free of charge (R0 per month) while the GitHub Team plan costs R64 per user per month (GitHub, 2026). (GitHub)

For a student development team this would likely be sufficient for the GitHub Free version.

6.7 Software Procurement Summary

Component	Proposed solution	Estimated initial monthly cost
Frontend hosting	Cloudflare Pages / Render	R0
Backend hosting	RENDER	Development: R0 + Production: R114+
Database	MongoDB Atlas	R0 initially
Map display	Mapbox GL JS	R0 within free tier
Routing	Mapbox Directions API	R0 within free tier
Geocoding	Mapbox Geocoding API	R0 within free tier
Source control	GitHub Free	R0
React / TypeScript / Node.js	Open-source	R0
Express / Mongoose / Axios / Tailwind	Open-source	R0
Domain name	Optional custom domain	Budget separately
SSL/TLS	Included with suitable cloud hosting	R0

Estimated initial operating cost

However, for a development/prototype deployment (using Mapbox and MongoDB free tiers, and free hosting services such as Heroku), the total estimated operating cost for this project would be R0 per month.

The initial budget for a small production implementation will range from R114.95/1st month + R89.95/month thereafter (shoulder top tier) to R600 +/month (MongoLab Large). This cost will include backend hosting, database configuration, API usage, and a custom domain name.

This is intended as a rough estimate of the running costs and is not a fixed quote.

6.8 Hardware Requirement

There is no need for the university to purchase a dedicated physical application server as the proposed architecture will run off cloud hosting for compute and database services as the primary production infrastructure.

The project may still require the following supporting infrastructure:

- Developer computers for development and testing.
- Existing university network infrastructure.
- Internet connectivity for cloud services and Mapbox.

Optional backup or storage infrastructure to create a backup of the application data.

No dedicated GPS hardware is required as the application will use the browser's Geolocation API.

Parking sensors / IoT hardware is not required for the current specified technology stack. However, if the final system requirements change and require automatic physical detection of parking-bay occupancy then this would need to be added to the stack.

7. External Interface Requirements

7.1 User Interfaces

UI Implementation and Technologies
The PukParking user interface will be implemented as a responsive Progressive Web App (PWA) using a component-based frontend architecture. The technologies are grouped below according to their role in implementing the PukParking interface and are presented in the recommended order of implementation and dependency.

7.1.1 Frontend Foundation and Application Structure

These technologies establish the core environment in which the PukParking interface will be developed.

React + TypeScript
React will be used as the primary frontend library for developing PukParking's user interface through reusable components.

React components will be used to construct the application's major interface elements, including the parking information screen, user-category selection, destination search, building history, interactive map, parking results, and navigation-related interfaces.

TypeScript (ES6+) will be used as the programming language for the React application.

Vite
Vite will be used as the project's development and build tool.
It will provide the development environment required to build, test, and package the React/TypeScript application efficiently.
Vite will also be used to generate the production build of the PukParking frontend.

7.1.2 Application Navigation and User Interaction

Once the core React application has been established, the following technologies will manage movement between PukParking's interface views and user input.

React Router
React Router will be used to manage client-side navigation between the different PukParking views without requiring full-page browser reloads.
It will support navigation between interfaces such as:

- Parking information
- Destination search and selection
- Building history
- Interactive campus map
- Parking-area results
- Navigation/directions

This allows PukParking to behave like an application while remaining accessible through the browser as a PWA.

React Hook Form
React Hook Form will be used for forms and structured user input within PukParking.
It can be used for interfaces where users need to provide or select information, such as:

- Selecting a User Category:
    - Admin
    - General User
- Entering a destination
- Searching for a building or destination
- Providing other required input where applicable

React Hook Form will handle input state, validation, and submission without requiring a separate form-management library.

7.1.3 Backend API and Application Data

These technologies will connect the PukParking interface to the application's backend services.

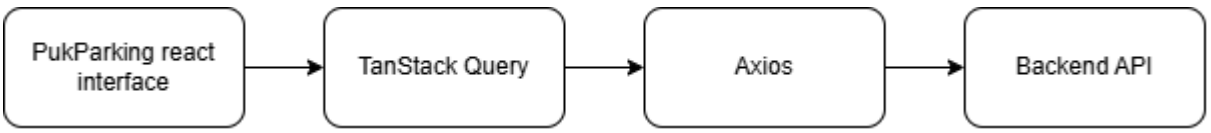
Axios + TanStack Query (React Query)
Axios and TanStack Query will be used together because they perform different but complementary functions.
Axios will handle communication between the PukParking frontend and backend APIs. It will be responsible for sending requests and receiving information such as:

- Building information
- Parking-area information
- Parking availability or status, where provided by the backend
- Destination information
- Route-related data
- User-category-specific parking information

TanStack Query (React Query) will manage the data received from these APIs. It will handle:

- Loading states while information is retrieved
- API errors
- Caching previously retrieved information
- Refetching data when necessary
- Keeping server-provided data synchronised with the interface

The relationship can therefore be represented as:



Axios is responsible for making the API request, while TanStack Query is responsible for managing the API data and its state within the application. Both technologies are therefore retained because they do not perform the same function.

7.1.4 Interactive Campus Map and Navigation Interface

The PukParking interactive campus map and navigation functionality will use the following technologies:

Mapbox GL JS

Mapbox GL JS will be used to display and interact with the PukParking campus map. It will display campus buildings, parking areas, the selected destination, the user's location, and calculated routes.

Mapbox Geocoding API

The Mapbox Geocoding API will be used to convert the user's entered or selected destination into geographic coordinates. This will allow PukParking to identify and display the selected building on the interactive map.

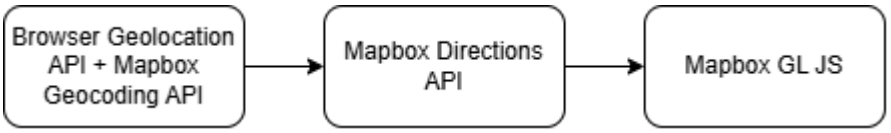
Browser Geolocation API

The Browser Geolocation API will be used to obtain the user's current location, with their permission. This location can be used as the starting point for route calculation and navigation.

Mapbox Directions API

The Mapbox Directions API will be used to calculate routes between the user's current location, relevant parking areas, and the selected destination. The resulting route will be displayed using Mapbox GL JS, including parking areas located close to or along the route where applicable.

The technologies will work together as follows:



This provides the location and destination information required to calculate and visually display the appropriate route and relevant parking areas.

7.1.5. Interface Components and Visual Feedback

Lucide React

Lucide React will provide the application's interface icons. It will be used for meaningful visual indicators such as:

- Search
- Location/current position
- Buildings
- Parking
- Navigation/directions
- Back and forward actions
- Menu controls
- Status indicators

A single icon library will be used to maintain visual consistency throughout the PukParking interface.

Framer Motion

Framer Motion will be used selectively for interface transitions and animations where they improve usability.

For example, it may be used for:

- Opening or closing interface panels
- Transitioning between parking results and map views
- Providing visual feedback when a destination is selected
- Smoothly displaying navigation-related interface elements

Animations will not be used excessively, particularly on mobile devices, where unnecessary animation could negatively affect usability or performance.

7.1.6. Progressive Web App Implementation

Vite PWA Plugin + Service Worker

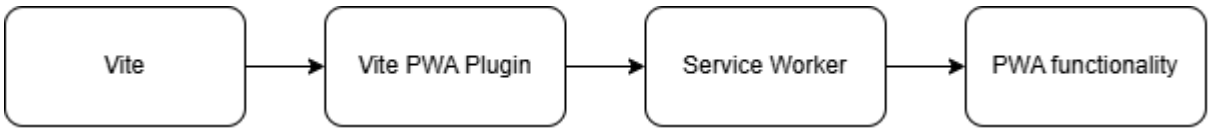
PukParking will be implemented as a Progressive Web App so that it can provide an app-like experience through supported web browsers.

The Vite PWA Plugin will be used to configure and integrate the PWA functionality into the Vite application, including:

- Web App Manifest configuration
- PWA installation support
- Service-worker integration
- Appropriate asset caching

The Service Worker will handle browser-level background functionality such as caching specified application resources and supporting appropriate offline behaviour.

The relationship is:



A separate custom service-worker implementation will not be introduced unless PukParking requires functionality that cannot be provided through the Vite PWA Plugin configuration.

7.1.7. Overall Frontend Technology Structure

The frontend technologies can therefore be viewed in the following implementation order:

1. Vite
2. React + TypeScript
3. React Router + React Hook Form
4. Axios + TanStack Query
5. Interactive Map Integration
6. Lucide React + Framer Motion
7. Vite PWA Plugin + Service Worker

This structure represents the relationship between the technologies rather than requiring them to be developed strictly one at a time. Development activities may overlap during implementation.

Technology Selection Principle

The PukParking frontend will avoid unnecessary duplication of technologies. Each selected technology should have a clearly defined responsibility within the system. Technologies that provide complementary functionality will be used together where necessary, while alternative libraries that perform substantially the same function will not be introduced without a specific project requirement.

Primary Design Philosophy

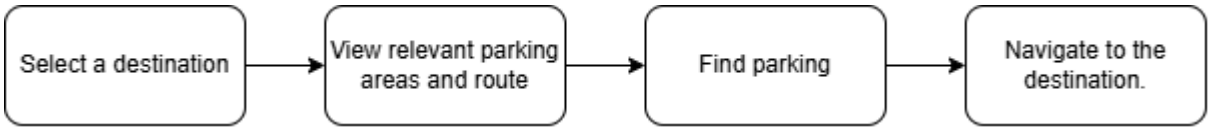
The primary design philosophy for PukParking is user-centred, mobile-first, and task-focused design. The interface shall prioritise simplicity, accessibility, clarity, and efficient completion of the user's primary parking-related tasks.

Design Principles

The PukParking user interface shall adhere to the following design principles:

- **User-centred:** The interface shall be designed around the needs of Staff, Students, and Guests.
- **Simple and intuitive:** The interface shall minimise unnecessary steps and make primary actions easy to understand.
- **Mobile-first and responsive:** The interface shall prioritise smartphone usability while supporting tablet and desktop screen sizes.
- **Clear visual hierarchy:** Important information such as destinations, parking areas, routes, and parking status shall be clearly distinguishable.
- **Consistency:** Common colours, typography, buttons, icons, spacing, and navigation patterns shall be used consistently.
- **Accessibility:** The interface shall provide readable text, sufficient colour contrast, clear labels, and appropriately sized interactive elements.
- **Map-focused:** The interactive campus map shall clearly communicate the relationship between the selected destination, route, buildings, and nearby or along-route parking areas.
- **Minimal cognitive load:** The interface shall avoid unnecessary menus, steps, and information that could make finding parking more difficult.
- **Feedback and visibility:** The system shall provide clear visual feedback for selections, loading states, errors, and completed actions.

The design philosophy ensures that the visual and interaction design supports PukParking's primary user journey:



7.2 Software Interfaces

7.2.1 Mapbox Interface

PukParking uses Mapbox GL JS for rendering the map, as well as the Mapbox Directions API and Mapbox Geocoding API for location searches and routing.

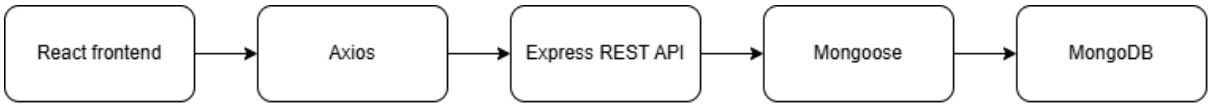
Mapbox GL JS is used for displaying the map inside the browser. For calculating walking routes, the Directions API from Mapbox is used and for searching locations by name or partial name the Geocoding API from Mapbox is used.

As previously noted, the PukParking application relies on Mapbox for all geographic and routing functionality, the credentials to access the relevant API will need to be protected. In line with existing practice for such sensitive configuration values, including database connections, API keys will be stored in dotenv files.

7.2.2 MongoDB Interface

On the server-side, MongoDB is a no-SQL database that is used to store various types of information including all the information about the parking zones. The backend uses Mongoose to interact with the MongoDB database. Information such as all the parking zones etc. is stored in the MongoDB database. All this information is semi-structured; hence MongoDB is used.

As previously stated, the frontend can't connect to the MongoDB database straight off the bat, the React frontend uses Axios to make requests to the Express REST API, which then communicates with MongoDB via the Mongoose interface to retrieve information from the relevant database.



This allows for Database Credentials to be kept safe on the backend while Database operations are facilitated via the configured Express Server.

7.2.3 Browser Interface

The front-end for PukParking is delivered in a browser to execute the compiled code for that application, and to provide browser-native code execution and functionality (such as use of the Geolocation API).

The user's current location can be used to centre the map as well as calculate proximity to all the parking areas within the system by making use of the Browser Geolocation API.

The Browser acts as the Interface to the user of the PukParking System.

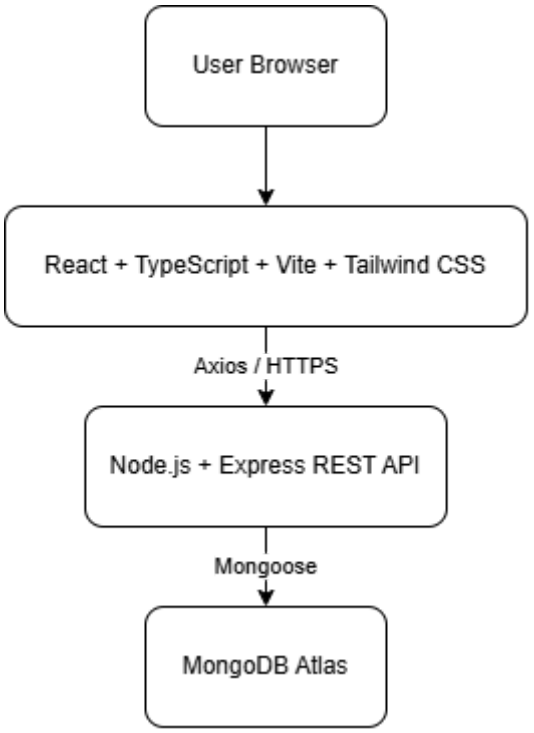
7.2.4 Hosting Platform Interface

The frontend and the backend can be hosted in two different cloud services, where the frontend is providing the interface to the user and the backend is hosting the REST API of the backend for the frontend to make calls to.

Render is a good choice to host the backend built with Express (Node.js application). Render supports Node.js and Express and allows for public web services. (Render)

A static frontend host like Cloudflare Pages could be used to host the React/Vite frontend. Cloudflare Pages supports deployment from a Git provider like GitHub and is globally distributed to deliver static content quickly to users. (Cloudflare Docs)

The resulting architecture can therefore be represented as:



The frontend application also communicates directly with Mapbox services for mapping, geocoding, and routing as identified in the required Mapbox services.

7.3 Communication Interfaces

The above-mentioned elements of PukParking communicate via standard web communication protocols. The frontend of PukParking communicates with the backend via HTTP-based REST API requests using Axios.

For the main path of communication within the application, HTTPS should be used for communication between the User's Browser and the application deployed on the Internet.

In terms of how your frontend makes API calls to your backend, these are HTTP-based REST API calls, which are made using the Axios technology as specified in the technology that you have chosen for the frontend.

The communication with MongoDB is realized by using the MongoDB database protocol. Mongoose is used to communicate with MongoDB in the Express Backend.

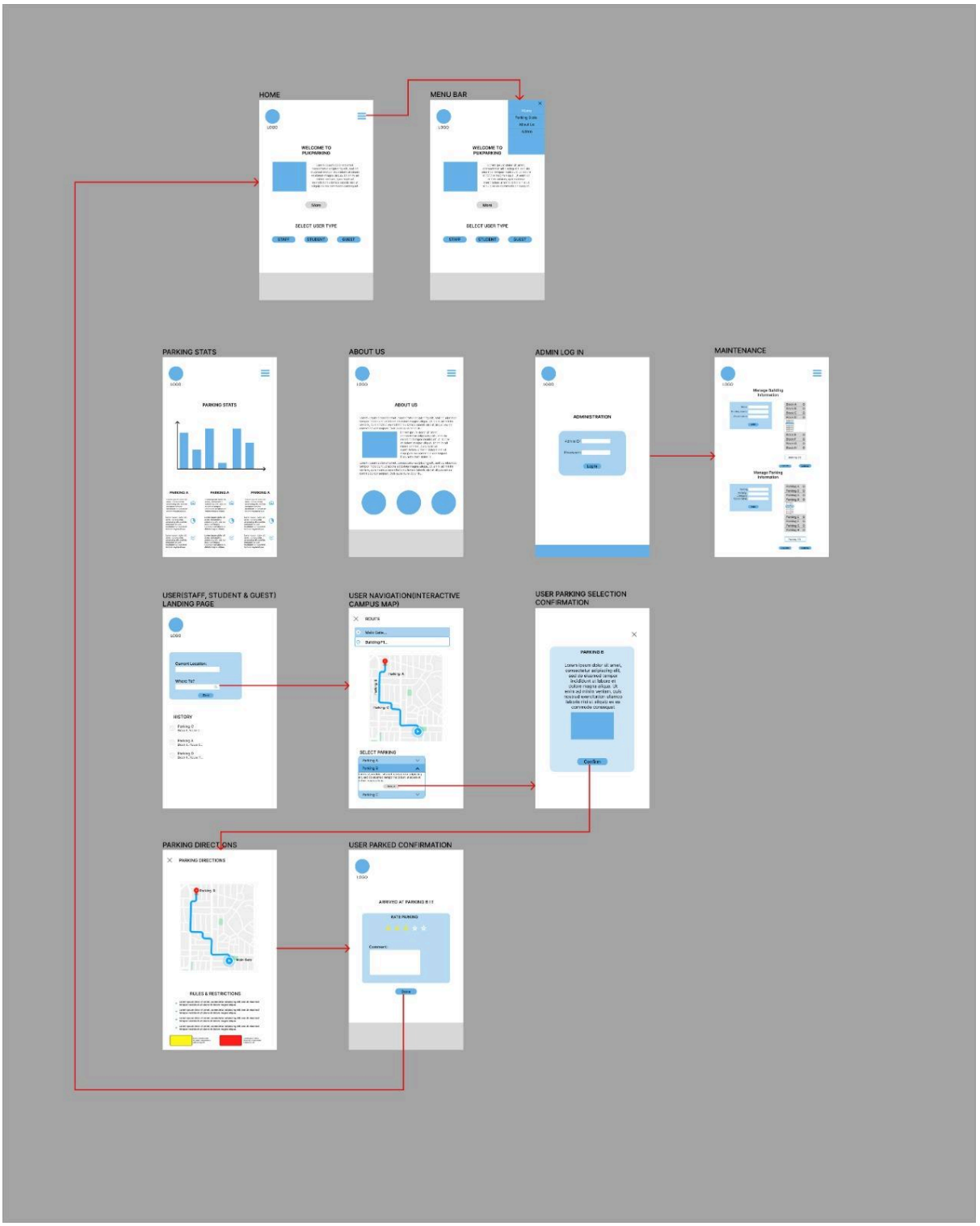
CORS: Since the frontend and the backend will most likely be hosted on two different servers, the communication between them will need to use Cross-Origin Resource Sharing (CORS) restrictions. PukParking API should enable CORS for the deployed frontend's origin only.
On the server-side we use the Helmet project to configure HTTP security headers, this will help prevent attacks like clickjacking and MIME-type sniffing.

The main communication interfaces are therefore:

Communication	Technology/Protocol	Purpose
Browser ↔ Frontend	HTTPS	Deliver and access PukParking
Frontend ↔ Backend	HTTPS/REST	Retrieve and submit application data
Frontend ↔ Mapbox	HTTPS/API	Maps, geocoding and routing
Backend↔ MongoDB	MongoDB connection	Store and retrieve application data
Browser ↔ Geolocation API	Browser API	Obtain user's current location
Frontend ↔ Service Worker	Browser API	Cache selected resources for PWA functionality

All communication will be done with security and reliability as top priorities. Communication of sensitive information (e.g. authentication) should be done securely. API keys should never be committed to the repository and should instead be added to environment variables. For production, all communication should be done over HTTPS.

8. Wireframes



9. References

Business Case For PukParking

Feasibility Case Study For PukParking

Cloudflare (2026a) *Cloudflare Pages documentation: Pricing*. Available at: Cloudflare Pages pricing documentation (Accessed: 30 August 2026).

GitHub (2026) *GitHub Pricing*. Available at: GitHub Pricing (Accessed: 30 August 2026).

Mapbox (2026a) *Mapbox GL JS pricing*. Available at: Mapbox GL JS pricing documentation (Accessed: 30 August 2026).

Mapbox (2026b) *Directions API*. Available at: Mapbox Directions API documentation (Accessed: 30 August 2026).

Mapbox (2026c) *Mapbox pricing*. Available at: Mapbox pricing (Accessed: 30 August 2026).

MongoDB (2026a) *MongoDB pricing*. Available at: MongoDB pricing (Accessed: 30 August 2026).

MongoDB (2026b) *Deploy a Free Cluster*. Available at: MongoDB Atlas Free Cluster documentation (Accessed: 30 August 2026).

Render (2026a) *Deploy for Free*. Available at: Render Free Services documentation (Accessed: 30 August 2026).

Render (2026b) *From side project to production: scaling your first app*. Available at: Render hosting and scaling guide (Accessed: 30 August 2026).

Render (2026c) *Web Services*. Available at: Render Web Services documentation (Accessed: 30 August 2026).